ePROJECT REPORT
FANIMATION
A Simple React Fan Catalogue Website
This report is based on the Fanimation project source code that was provided. It explains only what is in the project.

Table of Contents

Update this automatic table in Microsoft Word before printing: right-click it and choose Update Field.

Acknowledgements

We thank our lecturers and classmates for their support during this project. We also thank everyone who gave us advice while we were learning React and web development. This report explains our work in a clear and simple way.

1. eProject Synopsis

1.1 Introduction

Fanimation is a one-page website for showing fan products and accessories. A user can look through products, filter them, open product details, add items to a temporary cart, and use a contact form.

1.2 Purpose, Objectives and Scope

Show 15 fan and accessory products in a neat catalogue.
Let users filter by category, type, price, colour and brand.
Let users sort products, view details, save favourites and use a simple cart.
Show gallery, offers, about, FAQ and contact sections.
The project is front-end only. It has no login, database, payment, real checkout, product search bar, or message sending service.

1.3 Technologies Used

1.4 Expected Outcome

The finished website gives a simple shopping-style experience where visitors can discover Fanimation products without leaving the page.

2. eProject Analysis

2.1 Existing and Proposed System

There is no older system inside the uploaded files. The proposed system is the website that was built: a front-end product catalogue. It solves the problem of showing many fan types in one place with filters instead of making users check products one by one.

2.2 Requirements Found in the Code

2.3 Non-functional Requirements

The site is responsive, uses simple labelled form fields, has alt text on images, and uses Bootstrap plus custom CSS. It loads product data locally, so there is no API delay in the current version.

2.4 Assumptions, Constraints and Feasibility

Prices are stored as text like 95,000 and changed into numbers when needed. The cart and favourites are lost when the page refreshes. The project is easy to run as a demo because it does not need a server. For a real shop, it would need a backend, database and payment system.

3. eProject Design

3.1 Architecture

The browser opens index.html. main.jsx starts React and renders App.jsx. App puts all the page sections together and keeps shared cart and favourite data. ProductSection reads the product data and handles filters.

Figure 1. Simple system architecture

Mermaid diagram:

flowchart TD Browser --> main[src/main.jsx] --> App App --> Navbar & Hero & ProductSection & CartDrawer & Gallery & Offers & About & FAQ & Contact & Footer ProductSection --> ProductFilter & ProductGrid ProductGrid --> ProductCard --> ProductModal Data[(products.js)] --> ProductSection

3.2 Folder Structure

src/ components/ page parts such as Navbar, Gallery and Contact data/products.js 15 product objects and price helper assets/images/ logo, hero image and product images styles/global.css global page rules App.jsx main page and shared state main.jsx React start file

3.3 Navigation, State and Data Flow

Navbar and footer links scroll to sections. Category links send a small browser event to ProductSection so it can select the right category. App holds favouriteIds, cartItems and isCartOpen. ProductSection holds filter values. Other components keep their own small state, for example the gallery image and contact form.

3.4 Filtering and Modal Design

The filter checks category, type, colour, brand and maximum price. It then sorts by rating, newest product ID, or price. Clicking View Details opens a React modal. The modal closes with the close button, the background, or the Escape key.

Figure 2. Product filtering flow

Mermaid diagram:
flowchart TD User --> Filter[Choose filter or sort] Filter --> State[Update filter state] State --> Check[Check local product list] Check --> Sort[Sort matching products] Sort --> Grid[Show cards or no-products message]

3.5 Responsive Design and Styling

Bootstrap grid classes control most layouts. Product cards use three columns on large screens and two on medium screens. The gallery moves from four to three to two columns as the screen becomes smaller. The mobile navigation uses an overlay panel.

4. DFDs, Flowcharts and Process Diagrams

4.1 Context DFD

Mermaid diagram:
flowchart LR Visitor -->|clicks, filters, form input| Site[Fanimation React website] Site -->|products and feedback| Visitor Products[(Local products.js)] --> Site Site <--> Storage[(Browser storage for visitor count)]

4.2 Level 1 DFD

Mermaid diagram:
flowchart TD Visitor --> Navigation --> Sections Visitor --> Catalogue --> Products[(products.js)] Catalogue --> ProductCards Visitor --> CartFavorites --> AppState[(React state)] Visitor --> Contact --> Validation --> Message[On-screen success message]

4.3 Product details flowchart

Mermaid diagram:
flowchart TD Click[Click View Details] --> Modal[Open product modal] Modal --> Choice{Choose action} Choice -->|Add to Cart| Cart[Update cart and open drawer] Choice -->|Heart| Favourite[Update favourite list] Choice -->|Close / Escape / backdrop| End[Close modal]

4.4 Contact and full website process

Mermaid diagram:
flowchart TD Start --> Browse[Browse or filter products] Browse --> Details[View details or add to cart] Start --> Other[Gallery, offers, FAQ or about] Start --> Contact[Fill contact form] Contact --> Validate{Valid?} Validate -->|No| Errors[Show errors] Validate -->|Yes| Confirm[Show local confirmation] Confirm --> Note[No message is sent in this version]

5. Database Design / Structure

Current project: there is no database. Product data is inside products.js. The design below is only a future production idea.

5.1 Proposed ER Diagram

Mermaid diagram:

erDiagram CATEGORY ||--o{ PRODUCT : has PRODUCT ||--o{ REVIEW : receives USER ||--o{ REVIEW : writes USER ||--o{ WISHLIST : saves PRODUCT ||--o{ WISHLIST : is_saved USER ||--o{ ORDER : places ORDER ||--|{ ORDER_ITEM : contains PRODUCT ||--o{ ORDER_ITEM : ordered USER ||--o{ CONTACT_MESSAGE : sends

The future design is normalized because products, users, reviews and orders are stored in separate tables. This avoids repeating the same data many times.

6. Screenshot Placeholders

Take these screenshots after running the website. They are placeholders because the source upload did not include final screenshots.

Figure 1. Homepage / Hero Banner - Top bar, logo, hero text and Explore Collection button.

Figure 2. Products and Filters - Filter sidebar, cards, sorting and benefit box.

Figure 3. Product Modal and Cart - A product modal and open cart drawer.

Figure 4. Gallery and Offers - Gallery cards and offer cards.

Figure 5. FAQ, Contact and Footer - Accordion and contact form.

Figure 6. Mobile and Tablet View - Responsive view at small and medium screen sizes.

7. Source Code with Comments

The code already has useful short comments. These are the main parts a beginner should understand.

7.1 Cart logic

setCartItems((prev) => { const existing = prev.find((item) => item.id === normalizedProduct.id); if (existing) { return prev.map((item) => item.id === normalizedProduct.id ? { ...item, quantity: item.quantity + 1 } : item); } return [...prev, { ...normalizedProduct, quantity: 1 }]; });

This checks if the product is already in the cart. If it is, quantity goes up. If not, a new cart item is added.

7.2 Filter logic

if (filters.category && product.category !== filters.category) return false; if (filters.color && product.color !== filters.color) return false; if (filters.price && toNumber(product.price) > Number(filters.price)) return false;

A product is hidden when it does not match a selected filter. toNumber changes a formatted price into a number for comparison.

7.3 Contact validation

if (!values.name.trim()) nextErrors.name = "Please enter your name."; else if (!/^\S+@\S+\.\S+$/.test(values.email)) nextErrors.email = "Please enter a valid email address.";

The form checks for name, email and message before showing its local success message.

8. User Guide

9. Developer Guide

9.1 Start the Project

npm install npm run dev npm run build npm run lint

Use npm run dev while developing. Main dependencies are React, Vite, Bootstrap and Lucide React.

9.2 Add a Product

In src/data/products.js, add a new product object with a unique id, name, category, type, colour, price, rating, reviews, image and description. If it has oldPrice, it will also show in Offers.

9.3 Maintain and Extend

Keep styles in the correct component CSS file. Add a backend before adding real checkout or message sending. Add testing for filters, cart, modal and contact validation. Product search is not in the project yet and should be added as a new feature, not described as existing.

9.4 Build Note

During source checking, npm run build failed with a Vite/Rolldown emitted-name error and npm run lint reported a React effect warning in Navbar.jsx. These should be fixed before deployment.

10. Module Descriptions

11. Testing

There are no automated test files in the project. Manual browser testing on Chrome, Firefox and Edge is recommended after fixing build and lint problems.

12. Conclusion and Future Improvements

Fanimation meets its main goal as a simple front-end fan catalogue. It shows products clearly and gives useful interactions like filtering, modal details, favourites, cart, gallery, offers and contact validation. We learned how React components, props, state and CSS work together in one website.

Fix the build and lint issues.

Add a backend, database, login and real contact-message sending.

Save cart and favourites for users.

Add real checkout, payment and orders.

Add product search and automated tests.

Replace placeholders with group details and real screenshots before submission.

Group member | Student ID

Student 1: _____ | _____

Student 2: _____ | _____

Student 3: _____ | _____

Centre | _____

Course | _____

Semester / Session | _____

Submission date | _____

Tool | How it is used

React | Builds the user interface with components and state.

Vite | Runs and builds the React project.

Bootstrap | Helps with layout, forms, grid and FAQ accordion.

Lucide React | Shows icons.

CSS | Styles every component and makes the page responsive.

Requirement | Status

Navigation between page sections | Implemented with smooth in-page links.

Product catalogue | Implemented with 15 local product records.

Filters and sorting | Implemented.

Product details modal | Implemented.

Favourites and cart | Implemented but only kept in React memory.

Gallery, offers, FAQ, about and contact | Implemented.

Contact validation | Implemented; the form does not send data.

Real checkout, login, database and search | Not implemented.

Table | Important fields | Simple rules

Categories | category_id PK, name | Name should be unique.

Products | product_id PK, category_id FK, name, price, description | Price must not be negative.

Users | user_id PK, name, email | Email should be unique.

Reviews | review_id PK, product_id FK, user_id FK, rating | Rating should be 1 to 5.

Contact Messages | message_id PK, name, email, message | Save messages sent from the form.

Wishlist | user_id FK, product_id FK | One saved product per user.

Orders / Order Items | order_id PK; product_id FK; quantity; unit_price | Needed for real checkout.

[Screenshot placeholder] Homepage / Hero Banner

[Screenshot placeholder] Products and Filters
[Screenshot placeholder] Product Modal and Cart
[Screenshot placeholder] Gallery and Offers
[Screenshot placeholder] FAQ, Contact and Footer
[Screenshot placeholder] Mobile and Tablet View

Task	How to do it
Move around the site	Use the navigation bar, footer links or Explore Collection button.
Find a product	Choose a category or use the filters and Sort By menu.
See more information	Click View Details. Close with X, background click or Escape.
Save / add items	Use the heart for favourite and Add to Cart in the modal.
Change cart quantity	Open the cart icon and use minus or plus.
Contact the team	Enter name, valid email and message, then Send Message.
Important note	The cart, checkout and contact form are demo features only. Data is not saved or sent.

Module	What it does
App	Shows all sections and keeps shared cart/favourite state.
Navbar / Ticker / Hero	Top navigation, moving announcements and hero section.
ProductSection / ProductFilter / ProductGrid	Filters and displays the product catalogue.
ProductCard / ProductModal	Shows summary cards and detailed product pop-up.
CartDrawer	Shows cart items, quantities and subtotal.
Gallery / Offers	Shows product images and discounted products.
About / FAQ / Contact / Footer	Static company sections, FAQ accordion, validation form and footer.
products.js	Stores 15 product records and toNumber helper.

Test	Expected result	Result from review
Category filter	Only selected category appears.	Implemented in ProductSection filter logic.
Sort	Cards change order by rating, id or price.	Implemented.
Modal	Open and close correctly.	Implemented with Escape/backdrop support.
Cart	Add, increase, decrease and calculate subtotal.	Implemented in React state.
Contact	Show errors or local success message.	Implemented; no data sending.
Responsive layout	Page adapts to smaller screens.	CSS/Bootstrap rules found; full browser test still needed.
Build	Project should build.	Failed in checked environment; fix before deployment.
Lint	No errors.	One Navbar React-hooks error found.